

DSA Day 35: Minimum Spanning Tree

Book: Data Structures Through C in Depth - Ch7 §7.14 · pp 398-409 | Code in Python

Overview

MST connects all vertices with minimum total edge weight. Prim grows from a vertex; Kruskal adds cheapest edges using union-find.

Key concepts to master

- cut property
- Prim (priority queue)
- Kruskal (union-find)

Python implementation

```
def kruskal(n, edges):
    par=list(range(n))
    def find(x):
        while par[x]!=x: par[x]=par[par[x]]; x=par[x]
        return x
    mst=[]
    for w,u,v in sorted(edges):
        a,b=find(u),find(v)
        if a!=b: par[a]=b; mst.append((u,v,w))
    return mst
```

Complexity

Kruskal $O(E \log E)$; Prim $O(E \log V)$.

Common pitfalls

- Adding an edge that forms a cycle
- Union-find without path compression

Practice

- Find the MST both ways
- Which suits dense vs sparse graphs?

